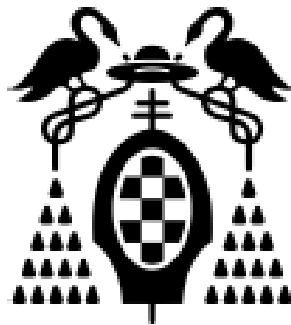


Computación Ubicua

PECL3 / PECL4

*Proyecto de desarrollo de frigorífico
inteligente*

Grado en Ingeniería Informática
Universidad de Alcalá



Isabel Barquilla Poyato
Sandra María Familiar Ferrer
Daniel Sas Morariu
Edgar Alexis Conforme Vacacela

Enero 2024-2025

Índice

Índice.....	2
Índice de Figuras.....	3
Vídeo demostrativo del proyecto:.....	4
1. Introducción.....	5
2. Objetivo del proyecto.....	5
3. Arquitectura del proyecto.....	8
3.1. Capa de procesado.....	9
3.1.1. Broker.....	10
3.1.2. Apache Tomcat.....	11
3.1.3. Base de datos.....	12
3.2. Capa de aplicación.....	25
3.2.1. Aplicación móvil.....	25
3.2.2. Aplicación Web.....	33
4. Conclusiones.....	35
5. Bibliografía.....	36
Anexo I – Manual de instalación.....	38
Anexo II – Manual de usuario de la aplicación Móvil.....	40
Anexo III – Manual de Usuario de la aplicación Web.....	49
Anexo IV – Simulación de datos.....	52

Índice de Figuras

Figura 1: Acceso a base de datos.....	13
Figura 2: Estructura de tablas.....	14
Figura 3: Estructura tabla producto.....	14
Figura 4: Estructura tabla usuario.....	15
Figura 4: Estructura tabla inventario.....	15
Figura 5: Estructura tabla sensores.....	16
Figura 6: Estructura tabla registro.....	16
Figura 7: Trigger cálculo de fecha de caducidad.....	16
Figura 8: LoadingActivity.....	26
Figura 9: MainActivity.....	27
Figura 10: LoginActivity.....	27
Figura 11: RegisterActivity.....	28
Figura 12: DashboardActivity.....	29
Figura 13: SensorActivity.....	30
Figura 14: statsActivity.....	31
Figura 15: CreateProductActivity.....	32
Figura 16: Diagrama de flujo de la aplicación móvil.....	33
Figura 17: Pantalla de carga.....	40
Figura 18: Pantalla principal.....	41
Figura 19: Pantalla de registro.....	42
Figura 20: Pantalla de login.....	42
Figura 21: Pantalla de inventario.....	44
Figura 22: Pantalla de medición de sensores.....	45
Figura 23: Pantalla de estadísticas.....	46
Figura 24: Pantalla de creación de producto.....	47
Figura 25: Pantalla inicial de la página web.....	50
Figura 26: Página web tras la carga de datos.....	51

Resumen

Este documento describe el desarrollo y la integración de un frigorífico inteligente como solución tecnológica para optimizar la gestión de alimentos en el hogar. El proyecto combina sensores de temperatura, humedad, agua, RFID y magnéticos para monitorizar el entorno interno del frigorífico, registrar el inventario, detectar anomalías y notificar al usuario en tiempo real. Estos sensores, conectados mediante la placa ESP32, permiten una comunicación eficiente con capas superiores a través del protocolo MQTT, integrándose con una aplicación móvil para una interacción fácil y efectiva.

La arquitectura del sistema está basada en cuatro capas: percepción, red, procesamiento y aplicación, las cuales trabajan en conjunto para garantizar la captura, transmisión y análisis de datos. Este enfoque modular asegura una alta escalabilidad y flexibilidad, permitiendo futuras mejoras en el sistema. Las pruebas realizadas verificaron la precisión y funcionalidad de los sensores, la estabilidad de la comunicación y la efectividad de las alertas generadas.

El proyecto no solo busca reducir el desperdicio alimentario y mejorar la eficiencia energética, sino también fomentar prácticas más sostenibles en el hogar, integrando tecnología accesible y eficiente para ofrecer una experiencia de usuario mejorada.

Palabras clave: Frigorífico inteligente, Sensores, RFID, ESP32, Computación Ubicua, Arquitectura de cuatro capas.

Vídeo demostrativo del proyecto:

- <https://drive.google.com/file/d/1BQnetgzB15vVNYM7aURfshGfQdM5xnq/view?usp=sharing>

1. Introducción

En la era actual de la transformación digital, los dispositivos inteligentes han dejado de ser una mera tendencia para convertirse en elementos clave en la gestión eficiente y sostenible de los recursos del hogar. Dentro de este marco, el frigorífico inteligente presentado en este proyecto aborda los retos comunes asociados a la conservación de alimentos, tales como el mal control de la temperatura, la acumulación de escarcha, los derrames inadvertidos, el olvido de puertas abiertas y la gestión ineficiente del inventario. La implementación de este sistema busca no solo mejorar la funcionalidad de un electrodoméstico esencial, sino también reducir el impacto ambiental y optimizar los hábitos de consumo en los hogares.

Este documento combina las fases de desarrollo correspondientes a la PECL3 y PECL4, centrándose en la integración avanzada de sensores, la comunicación eficiente con capas superiores mediante el protocolo MQTT y el diseño de una arquitectura modular basada en cuatro capas: percepción, red, procesamiento y aplicación. Durante estas fases, el sistema ha evolucionado desde su prototipo inicial hacia un modelo funcional que garantiza la captura, transmisión y análisis de datos en tiempo real, permitiendo la interacción del usuario a través de una aplicación móvil.

Además de detallar la arquitectura final, este documento incluye un análisis de las mejoras implementadas, como la optimización del flujo de datos entre los sensores y el servidor, así como la validación de las funcionalidades mediante pruebas exhaustivas. Estas pruebas han demostrado la capacidad del sistema para enviar alertas, registrar inventarios de productos a través de etiquetas RFID, y mantener un entorno controlado que optimice tanto la eficiencia energética como la conservación de alimentos.

En definitiva, el frigorífico inteligente se presenta como una solución tecnológica que responde a las necesidades de los hogares modernos, promoviendo una gestión más responsable y sostenible de los recursos. Este documento está estructurado en secciones que abarcan desde la arquitectura del sistema y las mejoras técnicas implementadas, hasta las conclusiones derivadas de las pruebas realizadas, proporcionando así una visión completa del estado actual del proyecto.

2. Objetivo del proyecto

El proyecto del frigorífico inteligente tiene como objetivo principal ofrecer una solución tecnológica integral que aborde los problemas comunes en la gestión de alimentos en el hogar. Estos incluyen la conservación inadecuada de productos debido al mal control de la temperatura, la falta de un seguimiento eficiente del inventario, el desperdicio alimentario, el consumo energético excesivo y el apoyo en la gestión de

una dieta saludable mediante el seguimiento de la calidad de los alimentos consumidos. A través de la integración de sensores avanzados y tecnologías de comunicación, se busca optimizar la experiencia del usuario, promoviendo hábitos más responsables y sostenibles.

El sistema incorpora sensores de temperatura y humedad, agua, magnéticos y RFID, que son gestionados mediante una placa ESP32, actuando como el núcleo de la capa de percepción. Estos sensores recopilan datos clave sobre las condiciones internas del frigorífico, así como información relacionada con los productos almacenados. La comunicación de estos datos hacia capas superiores se realiza a través del protocolo MQTT, permitiendo el procesamiento y la generación de alertas personalizadas que se transmiten al usuario mediante una aplicación móvil. De esta manera, el frigorífico inteligente se presenta como una herramienta accesible y eficaz para optimizar el manejo de los alimentos y la energía.

Funcionalidades clave

- Monitoreo constante de las condiciones internas: Los sensores de temperatura y humedad garantizan que los alimentos se conserven en un entorno óptimo, emitiendo alertas en caso de fluctuaciones críticas que puedan comprometer la frescura de los productos.
- Gestión avanzada del inventario y clasificación nutricional de los alimentos: El sistema RFID permite el registro automático de los productos almacenados, incluyendo detalles como fechas de caducidad, cantidades disponibles y su “NUTRI-SCORE”. Esto no solo facilita la planificación de compras, sino que también ayuda a reducir el desperdicio alimentario, además de fomentar hábitos alimenticios saludables.
- Detección de anomalías en tiempo real: Los sensores de agua y magnéticos detectan derrames, acumulación de escarcha y puertas abiertas, enviando notificaciones al usuario para prevenir problemas mayores.
- Optimización del consumo energético: Gracias a la detección temprana de puertas abiertas y otras ineficiencias, el sistema contribuye a minimizar el gasto energético, alineándose con prácticas sostenibles.

Alcance del proyecto

El proyecto abarca desde el diseño inicial y la selección de componentes hasta la construcción del prototipo funcional. Incluye la implementación de una arquitectura modular basada en cuatro capas: percepción, red, procesamiento y aplicación. Esto permite una buena integración entre los sensores, el servidor de procesamiento y la interfaz de usuario. Además, el desarrollo de una aplicación móvil facilita la interacción en tiempo real, permitiendo al usuario acceder a estadísticas, recibir alertas personalizadas y controlar el estado del frigorífico de forma intuitiva.

Este enfoque integral no solo mejora la gestión de los alimentos sino que también promueve un estilo de vida más sostenible y saludable, alineado con las tendencias actuales y las demandas de los consumidores modernos.

Enfoque en sostenibilidad y eficiencia

Debido a la creciente preocupación por la sostenibilidad, este proyecto busca reducir el impacto ambiental mediante:

- Ahorro energético: Mejorando la eficiencia operativa del frigorífico.
- Reducción del desperdicio alimentario: Al alertar sobre productos cercanos a su fecha de caducidad y condiciones internas desfavorables.
- Uso responsable de los recursos: Promoviendo hábitos que alineen las necesidades del usuario con prácticas sostenibles.

En definitiva, el frigorífico inteligente es una respuesta a las demandas de los hogares modernos, combinando tecnología accesible y de vanguardia con un diseño escalable y adaptable, capaz de evolucionar y responder a futuras necesidades.

CAMBIOS RESPECTO A LA PROPUESTA INICIAL

En comparación con la propuesta presentada en la PEC1, se han realizado los siguientes ajustes:

- Optimización de la capa de percepción: Durante las pruebas realizadas en la PECL2, se validaron las funcionalidades de los sensores, asegurando que cumplen con los objetivos del sistema, como el monitoreo de temperatura, humedad, detección de agua y el estado de la puerta. Los ajustes realizados se centraron en garantizar una integración fluida con la capa de red y la fiabilidad en la captura de datos para su transmisión al servidor.
- Refinamiento de la comunicación en capas superiores: Se optimizó el flujo de datos entre la ESP32 y el servidor MQTT para reducir la latencia y mejorar la estabilidad en la transmisión de mensajes, lo que refuerza la interacción entre el usuario y el sistema. Además, ahora los datos no se guardarán en Mqtt, como se hizo en la PECL2, sino que estarán en una base de datos específica que nos permitirá tener mayor capacidad de almacenamiento para gran variedad de productos.
- Incorporación del registro de la calidad nutricional de los alimentos: Se añadió una nueva funcionalidad para clasificar los alimentos según un sistema nutricional (A, B, C...) y registrar su consumo. Esto permite al usuario llevar un seguimiento de su dieta directamente desde la aplicación móvil, promoviendo así una alimentación más saludable y consciente.

En conjunto, estos cambios han sido implementados para mejorar la experiencia del usuario, garantizar la fiabilidad del sistema y asegurar que el frigorífico inteligente cumpla con su propósito de facilitar una gestión eficiente y sostenible de los alimentos en el hogar.

3. Arquitectura del proyecto

La arquitectura del frigorífico inteligente está diseñada para garantizar una integración eficiente y modular de todos los componentes, optimizando la captura, transmisión, procesamiento y presentación de datos en tiempo real. Este diseño modular se basa en cuatro capas principales: percepción, red, procesamiento y aplicación. A continuación, se describe cada capa y su contribución al sistema, así como los cambios realizados respecto a la propuesta inicial.

Descripción general de las capas

- Capa de percepción: Esta capa es responsable de capturar datos directamente del entorno interno del frigorífico mediante un conjunto de sensores:
 - Sensor de temperatura y humedad (DHT11): Monitorea las condiciones ambientales internas.
 - Sensor magnético: Detecta el estado de apertura o cierre de la puerta.
 - Sensor de agua: Identifica derrames y acumulación de escarcha.
 - Lector RFID (MFRC522): Facilita la gestión del inventario al identificar etiquetas de los productos almacenados.Todos los sensores están conectados a la placa ESP32, que centraliza la recopilación de datos y los transmite hacia la capa de red.
- Capa de red: Utiliza el protocolo MQTT para gestionar la comunicación en tiempo real entre la capa de percepción y las capas superiores. El broker MQTT actúa como intermediario para garantizar un flujo eficiente de datos, empleando un modelo de publicación/suscripción (publish/subscribe) que permite al sistema procesar y distribuir la información relevante a los usuarios y aplicaciones conectadas.
- Capa de procesamiento: Aquí se lleva a cabo la lógica principal del sistema:
 - Servidor web (Apache Tomcat): Gestiona las solicitudes provenientes de la capa de aplicación y coordina la interacción con la base de datos.
 - Base de datos (MariaDB): Almacena información clave sobre el inventario, alertas y condiciones internas del frigorífico.Este diseño asegura la trazabilidad y permite realizar análisis históricos para identificar patrones de uso y generar recomendaciones.
- Capa de aplicación: Es la interfaz principal para el usuario, representada por una aplicación móvil que proporciona:
 - Visualización del estado del frigorífico en tiempo real.
 - Gestión del inventario de productos mediante la identificación RFID.

- Recepción de alertas y recomendaciones personalizadas, como recordatorios de productos cercanos a su fecha de caducidad o anomalías detectadas en las condiciones internas.

La aplicación se comunica con el servidor mediante solicitudes HTTP para mostrar datos actualizados y garantizar una experiencia fluida para el usuario.

Interacción entre las capas

El flujo de datos en el sistema sigue una estructura claramente definida:

- Ascendente: Los datos recogidos por los sensores en la capa de percepción se envían a través de la capa de red al servidor, donde son procesados y almacenados.
- Descendente: Las alertas y recomendaciones generadas en la capa de procesamiento son enviadas a la capa de aplicación para su presentación al usuario.

Cambios respecto a la propuesta inicial

Durante el desarrollo del proyecto, se realizaron los siguientes ajustes para mejorar la funcionalidad y estabilidad del sistema:

- Mejoras en la capa de percepción: Se ajustaron los parámetros del sensor magnético y del lector RFID para aumentar su precisión y fiabilidad en entornos con interferencias.
- Optimización del flujo de datos: Se configuró el broker MQTT para gestionar reconexiones automáticas y garantizar la estabilidad de la comunicación.
- Simplificación de la arquitectura de la base de datos: Se redujo la complejidad inicial para priorizar la accesibilidad y la velocidad de respuesta. Ahora aquí es donde se almacenarán los datos de los productos y no en la placa, como sucedía en la PECL2.
- Rediseño de la interfaz de la aplicación móvil: Basándose en pruebas iniciales, se mejoró la claridad de la información y la facilidad de uso para los usuarios.

3.1. Capa de procesado

La capa de procesado es el núcleo del sistema encargado de recibir, almacenar, y gestionar la información proveniente de los sensores, así como de coordinar las acciones y generar las respuestas adecuadas hacia las capas superiores. En esta capa se realiza la lógica de negocio, el almacenamiento de datos, y la comunicación entre los distintos componentes del frigorífico inteligente. Los elementos que componen esta capa son el broker MQTT, el servidor Apache Tomcat y la base de datos

MariaDB, cada uno desempeñando un papel fundamental en el funcionamiento del sistema.

3.1.1. Broker

El Broker(MQTT) será el encargado de realizar la conexión servidor-cliente (publishers y suscribers) de manera que los clientes (suscribers) puedan recibir la información de una manera más clara y sencilla a partir de los *topics* a los que se hayan suscrito. Para poder hacer uso de dicho servidor, primero se deberá realizar una conexión (realizada antes de poder publicar o recibir los mensajes).

La información recibida por el servidor en los topics (en los que estamos suscritos) será mostrada. Dicha información como se explicó anteriormente, es enviada por los sensores que son los que obtienen valores; a partir de ahí, desde la capa de percepción la información es transmitida (por la ESP32) hacia la capa de red, siendo los sensores los publishers del “servidor” enviando los valores obtenidos a los topics. En ese instante en el que los publishers han acabado de enviar la información, el Broker enviará los mensajes (topics) a los suscriptores mostrando así la en la capa de red los valores que los clientes determinen.

Por ello, en este caso nuestros clientes que deseen utilizar las aplicaciones (Android o web) tendrán conexión con el Broker definido (MQTT) del cual visualizarán la información de los topics necesarios de la siguiente forma:

- frigorifico/temperature: información publicada por el sensor de temperatura a su respectivo topic cada vez que sea necesario y se actualice el valor.
- frigorifico/humidity: información publicada por sensor de humedad (DHT) a su respectivo topic ya definido al actualizarse el valor.
- frigorifico/door: información publicada en el topic por sensor de puerta (imán) cada vez que se actualice el valor del sensor (publisher)
- frigorifico/rfid: información publicada por el sensor rfid, con lógica que hará que el suscriptor pueda o insertar o eliminar valores según lo que el sensor haya leído (el cliente decidirá si quiere insertar o eliminar en la capa de aplicación).
- frigorifico/water: información enviada por el sensor de agua a su respectivo topic.
- frigorifico/anomalias: la información publicada a el siguiente topic necesitará la lógica por detrás de registrar cada vez que el valor de un sensor sea o distinto al deseado, o fuera de un rango definido; por ejemplo, si el sensor de temperatura registra una temperatura de 25 grados, y la deseada es menor a 20, se enviará al topic de anomalías la anomalía del sensor de temperatura que sobrepasa el límite o umbral definido. Lo mismo pasará con los demás sensores, como si está la puerta abierta, o hay demasiada humedad o se

detecta agua. De la siguiente manera el suscriptor podrá tener en cuenta varios aspectos y gestionará de otra manera su nevera.

3.1.2. Apache Tomcat

En este apartado, nos centraremos en la descripción de los Servlets y la lógica que hemos usado para su implementación con código en Java. Para la parte de instalación y configuración, se podrá obtener una descripción completa en el apartado de Manual de Instalación de este documento.

El paquete servlets contiene las clases necesarias para gestionar la interacción entre el servidor Tomcat y los clientes, ya sea a través de solicitudes HTTP o la integración con otros componentes del sistema, como la base de datos y MQTT. Estos servlets se encargan de procesar las peticiones entrantes, ejecutar la lógica de negocio necesaria y devolver respuestas adecuadas. Este paquete es uno de los que componen los códigos que contienen la lógica y entre los que también encontramos db o Logic, dichos paquetes se detallarán en apartados posteriores.

A continuación, se detallan las clases que componen este paquete y su contenido:

- CambiarModoServlet: Este servlet se utiliza para alternar entre los modos de funcionamiento del frigorífico (INSERTAR y ELIMINAR), almacenados en la variable estática DatabaseServlet.modos. Su método principal es doPost, el cual cambia el valor de DatabaseServlet.modos, devuelve el nuevo valor del modo como respuesta y permite controlar cómo interactúan los datos de entrada del sistema (por ejemplo, insertar o eliminar productos).
- DatabaseServlet: Es el servlet principal para acceder y gestionar la base de datos. Se utiliza para obtener datos de todas las tablas (productos, inventarios, usuarios, sensores, registros) y para integrar la lógica de MQTT.
Características especiales:
 - Se suscribe a tópicos de MQTT relacionados con el sistema del frigorífico.
 - Integra datos de múltiples tablas en un único objeto JSON que es enviado al cliente.Métodos principales:
 - doPost: Recupera los datos de la base de datos (productos, inventarios, usuarios, sensores y registros). Convierte los datos a formato JSON y los envía al cliente. Además, incluye la lógica de suscripción a tópicos MQTT.
 - doGet: Similar a doPost, pero con un enfoque en obtener los datos y devolverlos al cliente en formato JSON.
- ProductoServlet: Se encarga de gestionar operaciones relacionadas con los productos, como añadir un nuevo producto a la base de datos. Método principal: doPost: Recupera los parámetros del producto enviados desde el cliente (rfid_tag, nombre, nutri_score, plazo_caducidad). Crea un objeto

Producto y lo inserta en la base de datos utilizando el método insertarProducto de ConnectionDB. También devuelve un mensaje indicando el éxito o fallo de la operación. Esta clase se usa mucho, ya que gestionar productos es una funcionalidad clave del sistema.

- UsuarioServlet: Sirve para gestionar las operaciones relacionadas con los usuarios, como añadir o actualizar un usuario en la base de datos. Consiste en un método doPost, que se encarga de recuperar los parámetros del usuario (id, nombre, correo, pass, inventario) enviados desde el cliente. También inserta o actualiza el usuario en la base de datos utilizando el método insertarUsuario de ConnectionDB. Finalmente devuelve un mensaje confirmando el éxito o fallo de la operación.

Resumen:

- Servlets Clave: DatabaseServlet, CambiarModoServlet, ProductoServlet, y UsuarioServlet son las clases más importantes debido a su rol central en la lógica del sistema.

Interacción entre los Servlets

- Flujo de Datos entre Servlets y Componentes: DatabaseServlet actúa como un puente crítico entre la base de datos, MQTT y los servlets secundarios (CambiarModoServlet, ProductoServlet, UsuarioServlet, etc.). Esto permite que los datos actualizados se reflejen en todas las operaciones del sistema.
- Uso de MQTT: Varios servlets, como LogServlet, demuestran cómo el sistema está diseñado para ser reactivo y publicar eventos relevantes en tópicos MQTT. Esto añade modularidad y permite la extensión del sistema con otros dispositivos IoT en el futuro.
- Dependencias Compartidas: Componentes como MQTTBroker, ConnectionDB, y el paquete logic son fundamentales y utilizados de forma transversal por varios servlets, demostrando cohesión en el diseño del sistema.

3.1.3. Base de datos

En este proyecto, la base de datos juega un papel central, ya que es el repositorio encargado de almacenar toda la información necesaria para el funcionamiento del sistema. Esta información incluye datos sobre productos, usuarios, inventarios, sensores y registros de eventos. La integración con la base de datos permite garantizar la persistencia y la consulta eficiente de datos por parte de la aplicación Android y el servidor Tomcat.

La base de datos se implementa utilizando MariaDB, un sistema de gestión de bases de datos relacional que ofrece un buen rendimiento y soporte para integraciones con aplicaciones externas. Esta base de datos ya viene instalada en la máquina virtual que se nos proporciona y a la cual accedemos con los comandos especificados en la siguiente imagen:

```
ubicua@srvubicua:~$  
ubicua@srvubicua:~$ sudo mysql -u ubicua -p  
Enter password:  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 37  
Server version: 10.11.2-MariaDB-1 Ubuntu 23.04
```

Figura 1: Acceso a base de datos

El primer paso fue acceder al servidor MariaDB mediante la línea de comandos utilizando el usuario configurado para el proyecto. Desde allí, se creó una nueva base de datos llamada ubicua, que se emplea exclusivamente para almacenar la información del sistema. El comando utilizado para su creación fue: `CREATE DATABASE ubicua;`

Para trabajar con esta base de datos, se seleccionó mediante el comando: `USE ubicua;`

Para garantizar que tanto el servidor Tomcat como la aplicación Android pudieran interactuar con la base de datos, se configuraron los permisos necesarios para el usuario ubicua. Se logró con el siguiente comando: `GRANT ALL PRIVILEGES ON ubicua.* TO 'ubicua'@'%' IDENTIFIED BY 'contraseña';`

A continuación, se explicará el esquema de la base de datos y la división en tablas para el almacenamiento de datos:

ESTRUCTURA DE LA BASE DE DATOS

El diseño de la base de datos sigue un modelo Entidad-Relación (E/R), y su estructura refleja las necesidades específicas del sistema.

Esquema de tablas:

- Tabla productos: (rfid_tag, nombre, plazo_caducidad, nutri_score).
- Tabla sensores: (id, tipo, medicion, ult_lectura).
- Tabla alertas: (id, tipo_registro, id_producto, id_sensor, medicion, fecha).
- Tabla usuarios: (id, nombre, correo, contraseña, inventario).
- Tabla inventario: (id, producto_id, fecha_entrada, fecha_caducidad)

El diagrama E/R correspondiente a lo descrito anteriormente sería:

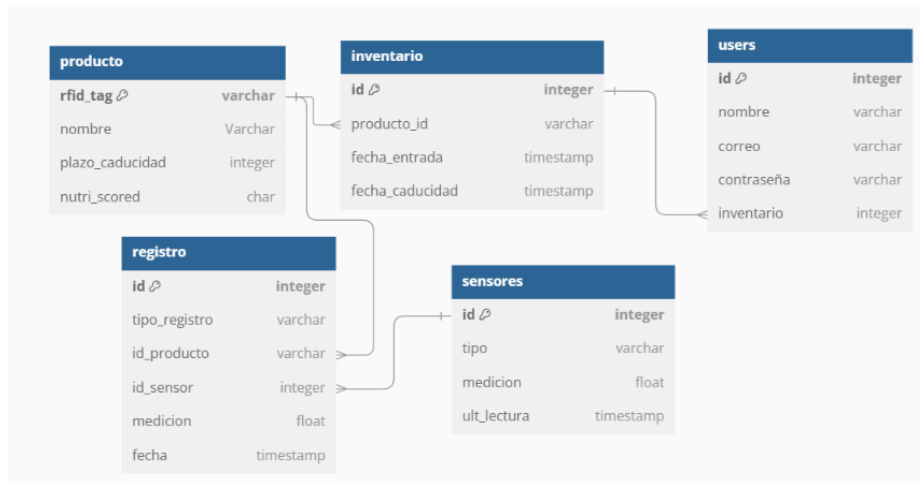


Figura 2: Estructura de tablas

A continuación analizaremos más en detalle cada una de las tablas:

- La tabla **producto** es la encargada de almacenar los datos relacionados con los productos registrados en el sistema. Cada producto se identifica de forma única mediante un código RFID, definido en el campo `rfid_tag`, que actúa como clave primaria. Además, incluye información esencial como el nombre del producto, el número de días que tarda en caducar (`plazo_caducidad`), y su Nutri-Score, que permite evaluar el valor nutricional del producto.

```

CREATE TABLE ubicua.producto (
  rfid_tag VARCHAR(255) PRIMARY KEY,
  nombre VARCHAR NOT NULL,
  plazo_caducidad INTEGER NOT NULL,
  nutri_score CHAR(1) NOT NULL
);

```

Figura 3: Estructura tabla producto

- La tabla **users** gestiona la información de los usuarios del sistema. Cada usuario tiene un identificador único (`id`), un nombre, un correo electrónico y una contraseña que garantizan la seguridad en el acceso. Adicionalmente, la tabla `users` se relaciona con la tabla `inventario` a través de una clave foránea que permite asociar cada usuario con los productos que almacena.

```
CREATE TABLE ubicua.users (
  id INTEGER AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(255) NOT NULL,
  correo VARCHAR(255) NOT NULL,
  contraseña VARCHAR(255) NOT NULL,
  inventario INTEGER,
  FOREIGN KEY (inventario) REFERENCES inventario(id)
);
```

Figura 4: Estructura tabla usuario

- La tabla **inventario** se utiliza para registrar los productos almacenados en el frigorífico. Cada registro de esta tabla incluye el identificador del producto correspondiente (producto_id), la fecha de entrada al inventario (fecha_entrada), y la fecha estimada de caducidad (fecha_caducidad). Esta última se calcula de forma automática mediante el trigger calculate_expiry_date, que utiliza el plazo de caducidad registrado en la tabla producto para determinar cuándo el producto dejará de ser apto para el consumo y el cual detallaremos más adelante.

```
CREATE TABLE ubicua.inventario (
  id INTEGER AUTO_INCREMENT PRIMARY KEY,
  producto_id VARCHAR(255) NOT NULL,
  fecha_entrada TIMESTAMP NOT NULL,
  fecha_caducidad TIMESTAMP NOT NULL,
  FOREIGN KEY (producto_id) REFERENCES producto(rfid_tag)
);
```

Figura 4: Estructura tabla inventario

- En cuanto a la tabla **sensores**, esta se encarga de almacenar información sobre los sensores conectados al sistema. Cada sensor está identificado por un campo único (id) y clasificado según su tipo (por ejemplo, temperatura o agua). Además, se registran datos como la última medición realizada (medicion) y la fecha de esa medición (ult_lectura), lo que permite llevar un seguimiento de las condiciones dentro del frigorífico.

```
CREATE TABLE ubicua.sensores (
  id INTEGER AUTO_INCREMENT PRIMARY KEY,
  tipo VARCHAR(255) NOT NULL,
  medicion FLOAT NOT NULL,
  ult_lectura TIMESTAMP NOT NULL
);
```

Figura 5: Estructura tabla sensores

- Por último, la tabla **registro** almacena el historial de eventos relacionados con los productos y los sensores. Cada registro incluye el tipo de evento (tipo_registro). Además, se relaciona con un producto (id_producto) o un sensor (id_sensor), y registra datos adicionales como la medición realizada y la fecha del evento. Esta tabla es clave para el seguimiento y control de los productos, así como para la generación de estadísticas y alertas en el sistema.

```
CREATE TABLE ubicua.registro (  
  id INTEGER AUTO_INCREMENT PRIMARY KEY,  
  tipo_registro VARCHAR(255) NOT NULL,  
  id_producto VARCHAR(255),  
  id_sensor INTEGER,  
  medicion FLOAT,  
  fecha TIMESTAMP NOT NULL,  
  FOREIGN KEY (id_producto) REFERENCES producto(rfid_tag),  
  FOREIGN KEY (id_sensor) REFERENCES sensores(id)  
);
```

Figura 6: Estructura tabla registro

Estas tablas están diseñadas con relaciones bien definidas que aseguran la integridad referencial y permiten que el sistema funcione de manera eficiente, tanto en términos de almacenamiento como de consulta de datos.

Además, en el código SQL resultante del diagrama anterior se ha añadido un trigger para la gestión de inserciones de registros en la tabla de inventario. Dicho trigger está destinado a el cálculo de la fecha de caducidad del producto como la suma de los días que aguanta dicho producto en buen estado a la fecha de entrada de este en el frigorífico.

```
CREATE TRIGGER calculate_expiry_date  
BEFORE INSERT ON inventario  
FOR EACH ROW  
BEGIN  
  DECLARE days_to_add INT;  
  -- Obtener el plazo de caducidad del producto referenciado  
  SELECT plazo_caducidad INTO days_to_add FROM producto WHERE rfid_tag = NEW.producto_id;  
  -- Calcular y establecer la fecha de caducidad  
  SET NEW.fecha_caducidad = DATE_ADD(NEW.fecha_entrada, INTERVAL days_to_add DAY);  
END//  
DELIMITER ;
```

Figura 7: Trigger cálculo de fecha de caducidad

Con la base de datos ya creada, se insertan los productos que van a encontrarse en el supermercado y que podrá contener por tanto nuestro frigorífico. Esto se realiza mediante el comando especificado a continuación:

```
INSERT INTO ubicua.producto (rfid_tag, plazo_caducidad, nutri_score) VALUES ('RFID001', 10, 'A', 'Leche Entera'),
```

//y se van especificando todos los posibles productos

PARTE LÓGICA DE LA BASE DE DATOS

Se ha realizado un código Java que permite la carga de datos en la base de datos y relaciona también el servidor Tomcat con la propia BBDD y la aplicación Android, de manera que los datos están siempre interconectados en todas las partes del proyecto y cualquier modificación permite su actualización instantánea.

A continuación, se detallan las partes más importantes:

Paquete db:

El paquete db se encarga de centralizar todas las operaciones necesarias para conectar y realizar consultas a la base de datos MariaDB. Este enfoque modular permite separar la lógica de acceso a datos del resto del sistema, garantizando que las clases relacionadas con la base de datos sean reutilizables y fáciles de mantener.

Las clases incluidas en el paquete son las siguientes:

- **Clase ConnectionDB:** La clase ConnectionDB es la encargada de manejar la conexión con la base de datos MariaDB. Además de establecer y cerrar conexiones, implementa métodos para gestionar transacciones y preparar sentencias SQL.

El propósito principal es establecer conexiones a través de un pool configurado en el servidor Tomcat, además de integrarse con el sistema MQTT para publicar y suscribirse a mensajes, e incluso proveer métodos utilitarios para crear sentencias preparadas (PreparedStatement) y realizar operaciones comunes con la base de datos.

Se describen algunos de sus principales métodos:

- **obtainConnection(boolean autoCommit):** Este método obtiene una conexión activa a la base de datos utilizando el pool configurado en Tomcat. Si la conexión falla, realiza hasta 5 intentos antes de devolver un error.

Flujo del método:

- Utiliza InitialContext para buscar el recurso DataSource configurado en Tomcat (definido como java:/comp/env/jdbc/ubica).
- Establece la conexión y la configura con el estado de autoCommit.
- Publica un mensaje en MQTT tras obtener el recurso (ctx.lookup), lo cual refuerza la integración con el sistema de mensajería.

- Si ocurre un error de tipo NamingException o SQLException, lo registra en los logs.

- closeTransaction(Connection con): Este método confirma una transacción activa en la base de datos utilizando commit(). Se invoca tras realizar una serie de operaciones exitosas en la base de datos para confirmar los cambios.
- cancelTransaction(Connection con): Revierte todas las operaciones realizadas en una transacción activa utilizando rollback(). Se invoca en caso de errores durante una transacción para garantizar que no se apliquen cambios parciales.
- closeConnection(Connection con): Cierra la conexión con la base de datos, liberando los recursos. Asegura que todas las conexiones abiertas sean cerradas correctamente para evitar fugas de memoria o saturación de recursos.

Métodos Específicos para la Lógica del Sistema:

- insertarProducto(Producto producto, Connection con): Inserta un nuevo producto en la tabla producto. Requiere un objeto Producto con detalles como rfid_tag, nombre, nutri_score, y plazo_caducidad.
- insertarUsuario(Usuario usuario, Connection con): Inserta un nuevo usuario o actualiza uno existente en la tabla users. En primer lugar, verifica si el usuario existe. Si existe, actualiza sus atributos. Si no existe, inserta el nuevo usuario.
- añadirProductoInventario(String rfidTag, Connection con): Este método permite registrar un producto en el inventario utilizando su identificador RFID. Además, realiza operaciones adicionales para calcular la fecha de caducidad y registrar la entrada en el historial de eventos del sistema. Es un ejemplo claro de cómo se conectan y trabajan en conjunto las tablas producto, inventario y registro.

Propósito del método:

- Verificar la existencia del producto en la base de datos.
- Calcular la fecha de caducidad del producto basándose en su plazo de caducidad.
- Insertar el producto en la tabla inventario con la fecha calculada.
- Registrar la operación en la tabla registro como una entrada.

Descripción del Flujo:

- Consulta del Plazo de Caducidad: El método utiliza un PreparedStatement para buscar el producto en la tabla producto y obtener su plazo de caducidad (plazo_caducidad). Si el producto no existe, el método devuelve false.

- Cálculo de la Fecha de Caducidad: A partir del plazo de caducidad (en días) y la fecha actual, se calcula la fecha de caducidad del producto.
 - Inserción en Inventario: Se inserta el producto en la tabla inventario, registrando la fecha de entrada y la fecha de caducidad calculada.
 - Registro del Evento: La operación se registra en la tabla registro como un evento de tipo "entrada", asociándolo al producto.
- eliminarProductoInventario(String rfidTag, Connection con): Este método elimina el producto más antiguo en el inventario utilizando su identificador RFID y registra la operación como una salida en la tabla registro.
 Descripción del flujo:
 - Busca el producto más antiguo en la tabla inventario basado en la fecha de entrada y lo elimina.
 - Registra la operación en la tabla registro como un evento de tipo "salida", especificando el producto eliminado.
 - añadirProductoRegistro(String tipo, String rfidTag, Connection con): Este método permite registrar un evento de entrada o salida de un producto en la tabla registro.
 Propósito del método:
 - Registrar en la tabla registro las entradas o salidas de productos del inventario.
 - Asociar cada evento con un producto específico utilizando su identificador RFID.
 Descripción del flujo:
 - Preparación de la consulta SQL:
 - Utiliza un objeto PreparedStatement para insertar un nuevo registro en la tabla registro. Los campos registrados incluyen:
 - tipo_registro: Indica si el evento es de tipo "entrada" o "salida".
 - id_producto: Identificador RFID del producto asociado al evento.
 - fecha: Se utiliza la fecha y hora actual como marca temporal para el evento.
 - Ejecución de la consulta: Ejecuta la consulta SQL para insertar el registro en la base de datos.
 - Gestión de errores: Si ocurre una excepción SQL durante la ejecución, registra el error en los logs mediante la clase Log y devuelve false.
 - añadirAlertaRegistro(int sensorId, float medición, Connection con): Registra una alerta en la tabla registro, asociándose con un sensor y

una medición. Es decir, Inserta una alerta en la tabla registro asociada a un sensor específico, incluyendo su medición.

- actualizarUltimaMedicion(int sensorId, float nuevaMedicion, Connection con): Esta función es llamada por la lógica del mqttSubscriber. Actualiza la última medición registrada por un sensor específico.
- insertarAnomalia(int sensorId, float medicion, Connection con): Inserta una anomalía en la tabla registro, marcándola como un evento de tipo "anomalía".

Métodos para Consultas y Estadísticas

- GetProductos(Connection con): Devuelve una lista de productos con atributos clave como rfid_tag, nombre, plazo_caducidad y nutri_score.
- GetInventario(Connection con): Recupera el inventario completo, incluyendo detalles como la fecha de entrada y la fecha de caducidad de cada producto.
- GetUsuarios(Connection con): Recupera una lista de usuarios registrados, incluyendo su correo, contraseña y el ID de su inventario asociado.
- GetSensores(Connection con): Devuelve información de todos los sensores almacenados en la base de datos, incluyendo su tipo, medición actual y última lectura.
- GetRegistros(Connection con): Recupera todos los registros desde la tabla registro, incluyendo tipo de registro, producto asociado, sensor asociado, medición y fecha.

Interacción con Mqtt

La clase ConnectionDB utiliza MQTTPublisher y MQTTSuscriber para publicar y suscribirse a mensajes de MQTT en eventos clave, como cuando se obtiene una conexión de base de datos. Por otro lado, la clase Topics sirve como modelo para gestionar los datos que circulan a través de estos tópicos MQTT.

Esta integración entre el acceso a la base de datos y la mensajería MQTT permite que el sistema sea dinámico y que las operaciones relacionadas con la base de datos puedan desencadenar eventos o notificaciones en tiempo real.

El paquete mqtt es fundamental en el sistema, ya que gestiona la comunicación mediante el protocolo MQTT, permitiendo la publicación y suscripción de mensajes entre los diferentes componentes del sistema. Este paquete está compuesto por tres clases principales: MQTTBroker, MQTTPublisher, y MQTTSuscriber.

A continuación, se desarrollan dichas clases, las cuales se encuentran en el paquete Mqtt de nuestro proyecto:

- Clase MQTTBroker: Esta clase se encarga de establecer la conexión principal con el broker MQTT.

Propósito:

- Proveer una conexión única y persistente al broker MQTT.
- Configurar los parámetros básicos necesarios para conectarse al broker, como credenciales y opciones de conexión.

Principales métodos:

- getInstance(): Método que implementa el patrón Singleton para asegurar que solo exista una instancia de MQTTBroker.
- Constructor: Configura las opciones de conexión (MqttConnectOptions), como usuario, contraseña y sesión limpia. Establece la conexión con el broker.
- Getters Estáticos: Métodos como getQos(), getBroker(), getClientId(), y getUsername() proporcionan información estática sobre la configuración del broker.

- Clase MQTTPublisher: La clase MQTTPublisher se utiliza para publicar mensajes en un tópic MQTT específico. Se basa en la configuración proporcionada por MQTTBroker.

Propósito:

- Enviar mensajes desde el sistema a un tópic MQTT.
- Facilitar la publicación de información relevante, como alertas o datos de sensores.

Principales métodos

- publish(MQTTBroker broker, String topic, String content): Establece una conexión con el broker utilizando las credenciales de MQTTBroker. Crea un mensaje MQTT con el contenido proporcionado y lo publica en el tópic especificado. Desconecta la conexión tras publicar el mensaje.

Flujo de Funcionamiento

- Conexión al broker.
- Creación del mensaje con una calidad de servicio (QoS) de nivel 2.
- Publicación del mensaje.
- Desconexión del cliente.

- Clase MQTTSuscriber: La clase MQTTSuscriber permite suscribirse a tópicos MQTT y manejar los mensajes recibidos. Implementa la interfaz MqttCallback, que define métodos para gestionar eventos relacionados con MQTT.

Propósito:

- Escuchar mensajes enviados a un tópic específico.
- Procesar los mensajes recibidos y realizar acciones según el contenido.

Principales métodos:

- suscribeTopic(MQTTBroker broker, String topic):
 - Establece una conexión con el broker MQTT y se suscribe al tópic especificado.

- Utiliza el objeto MQTTBroker para obtener credenciales y configurar la conexión.
- Define el objeto MQTTSuscriber como callback para manejar los mensajes entrantes.
- Logea el proceso de conexión y suscripción al tópic.
- Métodos de MqttCallback:
 - connectionLost(Throwable cause): Maneja eventos de pérdida de conexión con el broker. Aunque está definido, actualmente no realiza ninguna acción específica más allá del manejo interno.
 - messageArrived(String topic, MqttMessage message): Procesa los mensajes recibidos dependiendo del tópic mediante un bloque switch. Si el tópic recibido no es reconocido, lo registra como error en los logs. Cierra la conexión a la base de datos tras procesar el mensaje. Cada tópic está asociado a una funcionalidad concreta:
 - frigorifico/modo: Actualiza la variable modo en el servidor (DatabaseServlet.modos) con el valor recibido. Esto determina si los eventos de RFID deben insertar o eliminar productos del inventario.
 - frigorifico/temperature y frigorifico/humidity: Actualizan las lecturas de temperatura y humedad de los sensores en la base de datos a través de ConnectionDB.actualizarUltimaMedicion
 - frigorifico/door: Actualiza el estado de la puerta del frigorífico (abierta o cerrada) en la base de datos.
 - frigorifico/rfid: Gestiona las operaciones de entrada o salida de productos en el inventario según el estado de la variable modo (INSERTAR o ELIMINAR). Utiliza los métodos ConnectionDB.añadirProductoInventario y ConnectionDB.eliminarProductoInventario.
 - frigorifico/water: Registra si se detecta agua en el frigorífico.
 - frigorifico/anomalias: Gestiona anomalías como "Temperatura alta", "Humedad alta", "Puerta abierta", y "Agua detectada". Utiliza ConnectionDB.insertarAnomalia para registrar estas incidencias en la base de datos.
 - deliveryComplete(IMqttDeliveryToken token): Se invoca cuando un mensaje publicado ha sido entregado con éxito. En este caso, no se realiza ninguna acción adicional.

Partes destacadas en esta lógica (funciones importantes implementadas) a modo de resumen de lo anterior:

- Gestión del Modo de Operación: Se introduce la variable modo, que permite alternar entre dos estados:
 - INSERTAR: Los productos leídos por el sensor RFID se añaden al inventario.
 - ELIMINAR: Los productos leídos por el sensor RFID se eliminan del inventario.

La actualización del modo se realiza al procesar el tópico frigorifico/modo. El valor recibido en el mensaje MQTT se almacena en DatabaseServlet.modos y controla la lógica de operación en los tópicos relacionados con RFID.

- Gestión de Anomalías: Se añade soporte para procesar mensajes del tópico frigorifico/anomalias, que registra diferentes alertas en la base de datos según el tipo de incidencia detectada:
 - Temperatura alta: Se extrae el valor de temperatura del mensaje y se registra como una anomalía del sensor de temperatura.
 - Humedad alta: Similar al caso de temperatura, pero asociado al sensor de humedad.
 - Puerta abierta: Se registra como una anomalía del sensor de puerta si el mensaje contiene esta alerta.
 - Agua detectada: Registra una anomalía para el sensor de agua si el mensaje reporta la detección.

Todas las anomalías se gestionan mediante el método ConnectionDB.insertarAnomalia.

- Gestión de Sensores y Mediciones: Los tópicos frigorifico/temperature, frigorifico/humidity, y frigorifico/door permiten actualizar las lecturas de los sensores en la base de datos mediante el método ConnectionDB.actualizarUltimaMedicion. La lógica identifica el sensor correspondiente según el tópico y actualiza su estado con el valor recibido en el mensaje.
- Gestión del Inventario: El tópico frigorifico/rfid gestiona las operaciones del inventario dependiendo del estado del modo:
 - INSERTAR: Se añade el producto al inventario utilizando ConnectionDB.añadirProductoInventario.
 - ELIMINAR: Se elimina el producto más antiguo del inventario utilizando ConnectionDB.eliminarProductoInventario.
- Gestión del Sensor de Agua: El tópico frigorifico/water actualiza el estado del sensor de agua en la base de datos

según el valor recibido: Si el mensaje contiene "Agua detectada", se actualiza el sensor con el valor correspondiente.

LÓGICA DE DISTRIBUCIÓN DE LOS DATOS EN TABLAS

Aunque anteriormente ya se ha realizado un resumen de cómo se distribuyen los datos en diferentes paquetes, en esta parte se detallarán en mayor profundidad los códigos que contienen la lógica mencionada.

Todas las clases creadas de lógica de estructuras de datos, se encuentran en el paquete de Logic. Actúa como un puente entre la clase ConnectionDb y la interacción del sistema con los usuarios y dispositivos (sensores, productos, etc.). Este paquete agrupa clases que manejan diferentes aspectos del sistema, como el inventario, los usuarios, los sensores y los registros. A continuación, se detallan todas sus clases:

- Clase Inventario: Esta clase gestiona la lógica del inventario del frigorífico, incluyendo las operaciones de añadir y eliminar productos. Está estrechamente vinculada con la tabla inventario y la tabla registro de la base de datos.
- Clase Log: La clase Log actúa como una capa de registro de eventos. Usa la biblioteca Log4j para registrar diferentes tipos de eventos en archivos o sistemas de logs. Cada operación crítica, como una conexión fallida o una inserción en la base de datos, queda registrada con un nivel de detalle que facilita la depuración y el análisis del sistema.

Tipos de Logs:

- log: Registra eventos generales del sistema.
 - logdb: Registra eventos relacionados con las operaciones de base de datos.
 - logmqtt: Registra eventos relacionados con la comunicación MQTT.
- Clase Producto: La clase Producto encapsula la estructura relacionada con los productos del frigorífico.
 - Clase Projectinitializer: Esta clase es un listener que se ejecuta al inicializar el servidor Tomcat. No tiene funciones relevantes en nuestro proyecto
 - Clase Usuario: contiene la información de la estructura de un objeto Usuario con sus atributos.

El paquete Logic encapsula la lógica de negocio del sistema de manera eficiente y modular. Cada clase tiene un propósito claro y métodos bien definidos para interactuar con la base de datos, sensores, productos y usuarios. Este diseño asegura que el sistema sea escalable, mantenible y fácil de integrar con otros componentes.

3.2. Capa de aplicación

El objetivo de esta capa es el mismo que el explicado en la anterior entrega, es decir, crear una aplicación con una interfaz accesible al usuario donde este pueda consultar los datos actualizados de su frigorífico. Además, también se podrá mandar información a la capa de procesamiento de cara a la creación de usuarios y productos.

3.2.1. Aplicación móvil

Para la creación de la aplicación hemos usado AndroidStudio; generamos el proyecto SmartFridge que contendrá:

- Archivos .java: son aquellos archivos que comprenden el código de la lógica necesaria para el funcionamiento de la aplicación a través de las interfaces, definiendo el flujo de operaciones a ejecutar al interactuar con las mismas.
- Archivos .xml: son las interfaces creadas para que el usuario pueda interactuar con la aplicación de forma más sencilla y accesible.
- Archivos para el build tanto gradle como manifest que son los encargados de estructurar la aplicación, añadiendo librerías, archivos que se ejecutarán etc...
- Archivos de imagen necesarios para la aplicación, como el logo de la misma o el icono de notificaciones, de cara a hacer la aplicación más intuitiva y atractiva visualmente.

→ Archivo.java e interfaz asociada

En este apartado procederemos a enumerar y explicar el funcionamiento de las interfaces creadas para la práctica.

- LoadingActivity.java: Esta actividad es la primera que se muestra al ejecutar la aplicación. En esta pantalla se dará la bienvenida al usuario y se mostrará un icono de carga que durará 3 segundos para indicar al usuario que se está cargando el servicio de la aplicación.



Bienvenido a SmartFridge



Figura 8: LoadingActivity

- MainActivity.java: es la pantalla de inicio, donde se recibe al usuario, se le muestra el logo del proyecto y se le presentan 2 botones, uno para iniciar sesión y otro para registrarse si no tiene usuario registrado.

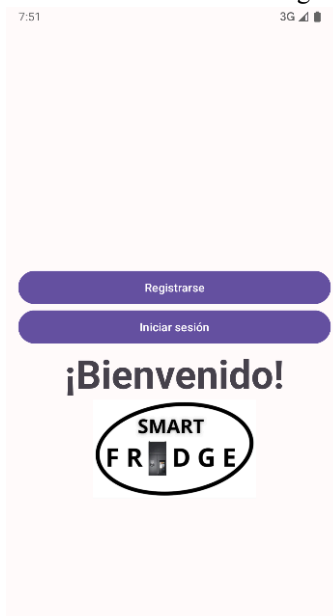


Figura 9: MainActivity

- LoginActivity.java: interfaz que se le muestra al usuario en caso de que este haya pulsado el botón Iniciar Sesión, donde se le pedirá el usuario (correo del usuario) y la contraseña decidida por el usuario tras haberse registrado. Una vez introducidos el usuario y la contraseña, se tratará de acceder al Dashboard o interfaz principal de la aplicación donde ya se muestra la nevera con el inventario, los productos, y las estadísticas y valores recogidos por los sensores. Si son correctos el usuario y la contraseña, se accederá de forma exitosa a la siguiente interfaz, pero si no, se borrarán los campos completados (usuario y contraseña) y se mostrará un mensaje de aviso (usuario o contraseña inválidos). Para esto se conectará con los servlets para obtener el listado de los usuarios dados de alta en la base de datos.

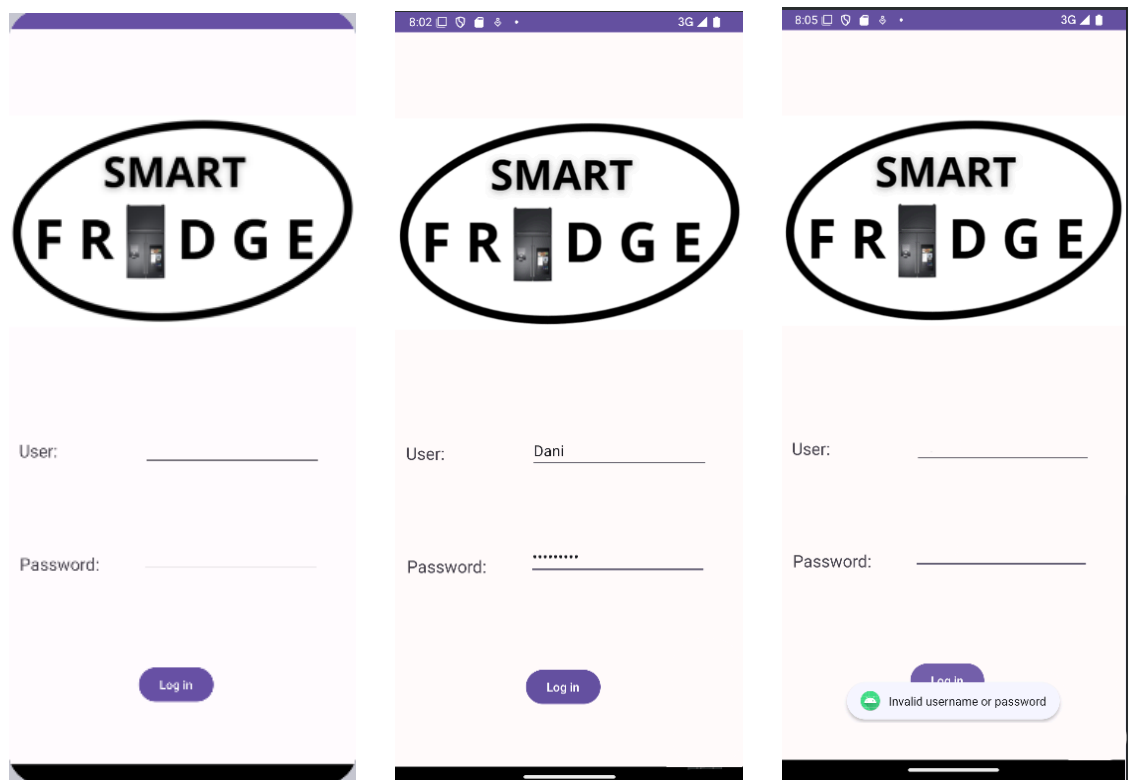


Figura 10: LoginActivity

- RegisterActivity.java: si el usuario no tiene cuenta, puede crear una cuenta clicando en el botón Registrarse. Este, le llevará a una interfaz en la cuál se deberán completar los campos nombre, contraseña y mail de forma obligatoria (si uno de estos campos no se completa, no se podrá registrar).

También si la contraseña es menor a 8 caracteres, no será válida. Finalmente, si se registra correctamente, se le mostrará un mensaje diciendo “Registrado con éxito”, y podrá pasar directamente a la pantalla de inicio de sesión (interfaz de LoginActivity) para poder entrar a la interfaz Dashboard donde se gestiona la nevera. En caso contrario, se le mostrará un mensaje para que el usuario sepa que no se ha podido registrar. Para hacer el registro efectivo deberá conectarse con los servlets para enviar la información del nuevo usuario a la base de datos.

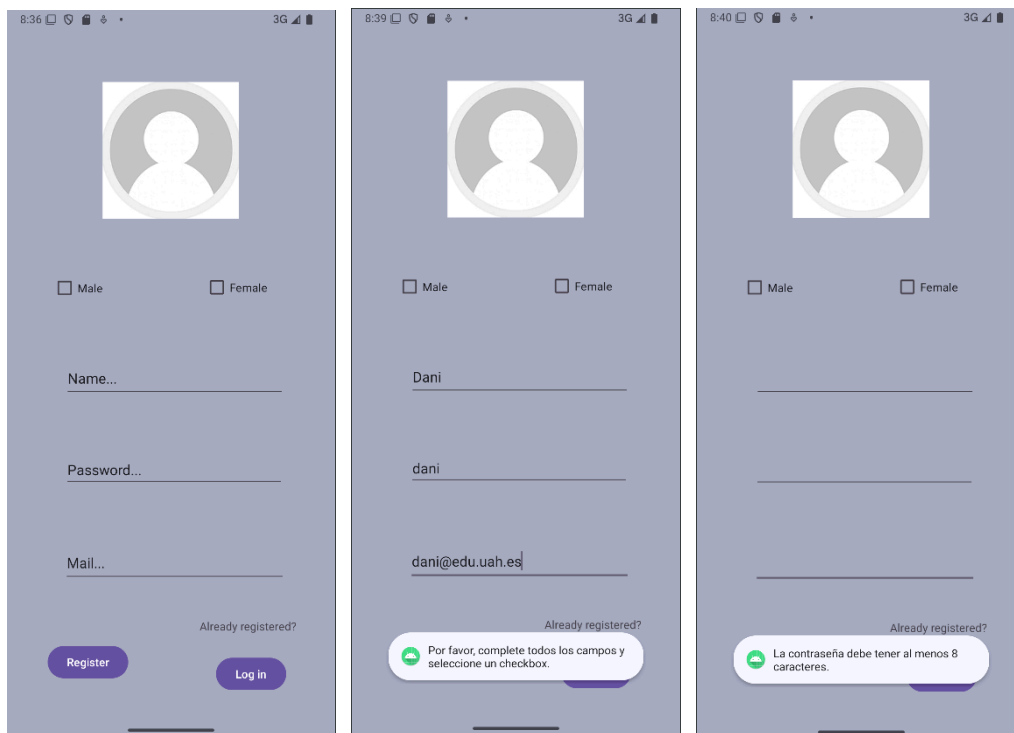


Figura 11: RegisterActivity

- DashboardActivity.java: código que gestiona la interfaz donde se mostrará el inventario del frigorífico (productos en la nevera en el momento actual, así como sus fechas de caducidad), así como varios botones con las acciones posibles que puede realizar el usuario. En primer lugar encontramos un botón “Medidas Sensores” el cuál en tiempo real mostrará los valores de los sensores de la nevera; continuamos con el botón “Estadísticas consumo” que conduce a otra nueva interfaz en la cual podremos ver las estadísticas de los últimos consumos hechos por el usuario, así como de las alertas que haya habido desde entonces. Finalmente habrá otro botón “Crear Producto” que permitirá al usuario registrar productos en la base de datos para

posteriormente poder registrarlos en el inventario. Además, podremos definir el modo en el que este nuestra nevera, pudiendo activar el modo de entrada de alimentos (los alimentos que detecte el RFID se añadirán al inventario) o desactivandolo (los alimentos detectados saldrán del inventario). Por último encontramos un botón de “Cerrar sesión” que nos devolverá a la interfaz de inicio MainActivity.java. Además, podremos ver en la esquina superior derecha una campana que avisará al usuario si tiene alguna alerta o notificación sobre la nevera o el inventario (registros). Tanto para la carga inicial de los productos como la actualización a tiempo real de las notificaciones de alertas nos conectaremos con los servlets desarrollados pidiéndole la información correspondiente, al igual que mandaremos información a los mismos para poder modificar el estado del modo de entrada de alimentos.

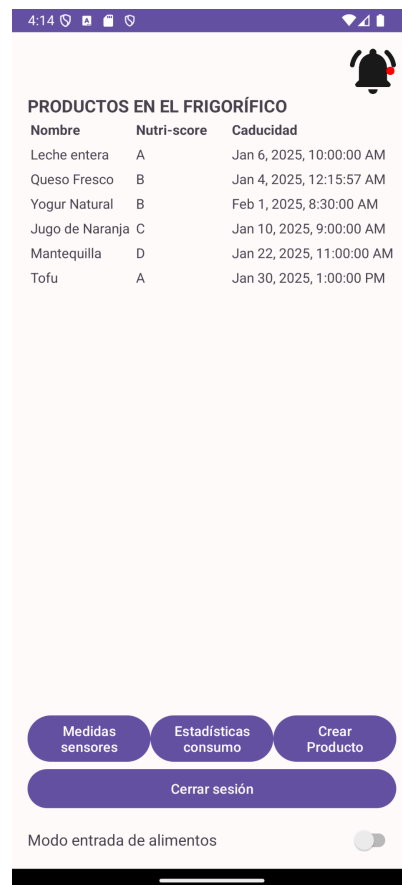


Figura 12: DashboardActivity

- **SensorActivity.java:** Código que gestiona la interfaz donde mostraremos los datos medidos por los sensores de agua, temperatura, humedad y puerta abierta en tiempo real. De esta forma, podemos controlar el buen estado del frigorífico desde la misma aplicación. Además, se añade un botón en la parte inferior para poder volver a la pantalla anterior y un texto informativo sobre la información mostrada. Para obtener los datos de los sensores en tiempo real se conectará a los servlets desarrollados cada dos segundos, obteniendo la información actualizada de los mismos.

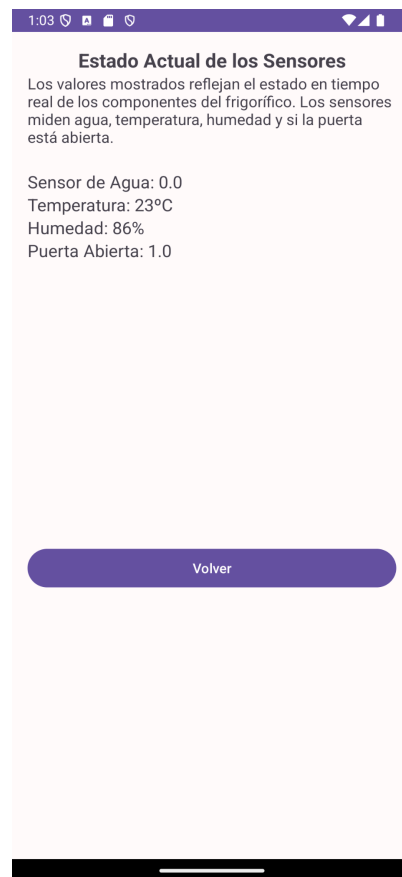


Figura 13: SensorActivity

- **statsActivity:** código que gestiona la interfaz en la cual se muestran las estadísticas más importantes sobre el inventario de la nevera al usuario y las alertas recibidas. De esta forma al entrar a la interfaz podremos usar el botón superior "Insertar fecha para cálculo de estadísticas" para introducir desde que día queremos recoger los datos de consumo y alertas. Esto servirá al usuario para que pueda visualizar si ha mejorado sus hábitos de consumo y

buen estado de su frigorífico o, por el contrario, ha empeorado. Para esto se mostrará un calendario donde el usuario podrá seleccionar la fecha de inicio de periodo de forma cómoda y visual y tras esto se realizan y mostrarán los cálculos del consumo de alimentos desde entonces (% de productos consumidos de cada nutri-score), así como el número de alertas que se hayan producido desde entonces. Nuevamente contaremos con un botón “Volver” en la parte inferior de la pantalla que nos devolverá a la interfaz anterior. Nuevamente nos deberemos conectar al servlet correspondiente para obtener la información necesaria.

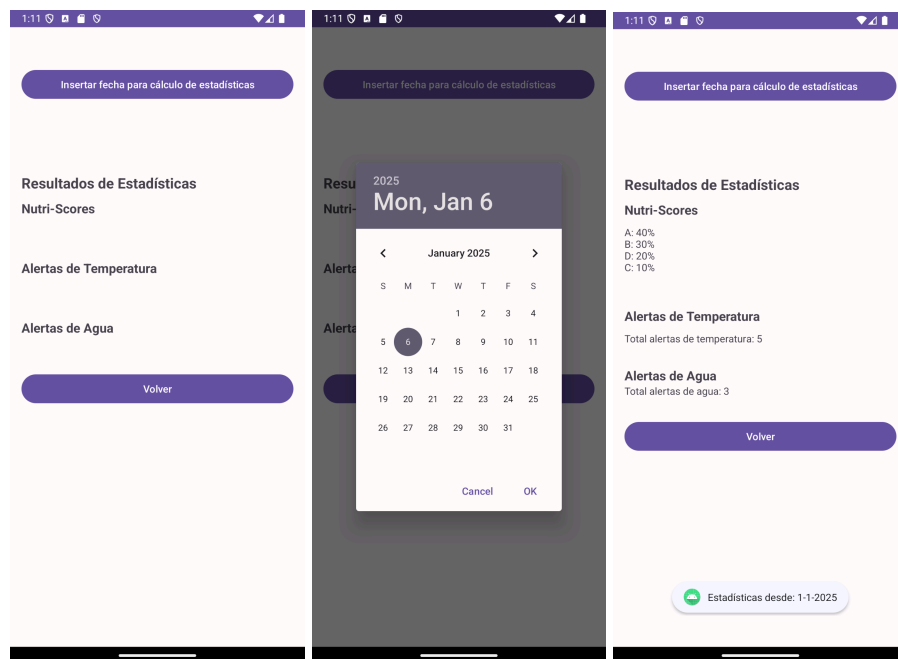
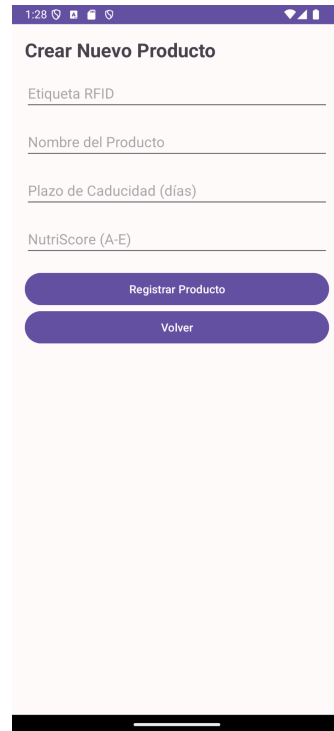


Figura 14: statsActivity

- **CreateProductActivity.java:** Código que gestiona la interfaz de creación de productos. Esta nos dará la posibilidad de crear un nuevo producto en la tabla correspondiente a la base de datos, de cara a poder introducir en la nevera productos que no existían o no se tenía constancia de ellos en un inicio. De hecho está pensado de forma que el usuario pueda pegar etiquetas a RFID incluso a comidas cocinadas por él, por lo que no estarían en la base de datos en un inicio, posibilitando tener en cuenta este tipo de alimentos a la hora de cálculo de estadísticas de consumo y similar. Para el registro, sólo tendremos que introducir los datos necesarios y pulsar el botón “Registrar producto”. Por último, contaremos con un botón “Volver” con el que podremos volver a la interfaz anterior.

Para poder registrar el producto en la base de datos nos conectaremos al servlet desarrollado para tal efecto.



1:28

Crear Nuevo Producto

Etiqueta RFID

Nombre del Producto

Plazo de Caducidad (días)

NutriScore (A-E)

Registrar Producto

Volver

Figura 15: CreateProductActivity

A continuación y para facilitar la comprensión del funcionamiento de la aplicación, se muestra un diagrama de flujo de las actividades de la aplicación.

web mientras que doGet es utilizado para Android. Las funciones en general tratan sobre actualizar o recuperar datos desde la base de datos (tales como generar producto e insertarlo a la base de datos mediante post o recibir la base de datos para mostrarla en el frontend mediante get).

- **Conexión al Broker MQTT:** Al iniciar, doGet se conecta al broker MQTT, suscribiéndose a todos los tópicos relevantes para recibir actualizaciones en tiempo real. Esto permite que la aplicación refleje el estado actual del sistema sin intervención manual.
- **Servlets addProducto y addUsuario:** Estos servlets gestionan la inserción de nuevos productos y usuarios en la base de datos. Cada uno de estos métodos es llamado de la misma manera que se explicó anteriormente, indicando el tipo ("POST") y definiendo la lógica que permitirá que en la comunicación servidor-base de datos este pueda insertar dentro de la base de datos la información que el usuario determine necesaria.

Frontend-index.html, interfaz, interacción con usuario:

Algunas de las funcionalidades principales relacionadas al frontend son:

- Envía solicitudes HTTP a las rutas definidas en el backend.
- Usa respuestas JSON para actualizar dinámicamente las tablas y otros elementos de la página.
- Controla la experiencia del usuario.

Para ello, es muy importante conocer las funciones y componentes que permite que esto sea así

- *Botones inicio, parada y modo del lector rfid:* botones con distintas funcionalidades para iniciar y detener la actualización automática de datos (Start Interval y Stop Interval), así como para cambiar el modo operativo del lector (Cambiar Modo) pudiendo así introducir o sacar productos del frigorífico (permitiéndole cierta libertad al usuario) enviando a los servlets cambiar modo (lógica para cambiar entre los modos insertar y eliminar).
- *Botón de añadir producto:* utilizando campos de entrada en los cuales el usuario podrá introducir el id del producto (leído por el lector rfid llamado "rfid tag"), el nombre, el nutriscore y la fecha de caducidad, que se mapean a solicitudes POST enviadas a los servlets correspondientes.
- Función principal databaseAction que es la que llama al servlet que contiene toda la lógica con el funcionamiento de la aplicación web obtenida gracias a la url ("/ServerExampleUbiComp-1.0-SNAPSHOT/databaseAction"), y desde la cual se tendrán en cuenta tanto la conexión de la base de datos, como la gestión de todas las tablas que se muestran en la interfaz y los datos alojados en la base de datos. Además de esto, pueden modificar la base de datos gracias a funciones que lo permiten de tipo get o post realizando un update; de forma sencilla, se prepara un objeto preparedStatement, que contiene las queries necesarias para poder modificar la base de datos, y estos

son enviados a la base de datos siendo los responsables de añadir, o modificar la base de datos (que son mostrados por tabla).

Funcionamiento Detallado:

Cuando se accede al index.html, el usuario puede interactuar con el sistema de las siguientes maneras:

- Iniciar la recopilación de datos: Al pulsar "Start Interval", la página comienza a actualizar los datos mostrados cada 7 segundos mediante la función databaseAction definida en el script. Esto envía solicitudes regulares al servlet que gestiona la recuperación y muestra de los datos actuales.
- Gestión de datos: Los formularios permiten a los usuarios ingresar nuevos productos y usuarios, que se envían al servidor para su procesamiento y almacenamiento.
- Monitorización en tiempo real: Los datos de productos, inventario, usuarios y sensores se actualizan y muestran en tablas en la interfaz, permitiendo a los usuarios ver el estado actual del sistema en cualquier momento.
- Control de operaciones: Los botones como "Cambiar Modo" permiten al usuario ajustar cómo el sistema responde a las entradas de los sensores y maneja los productos en el inventario.

4. Conclusiones

El desarrollo de este proyecto ha permitido implementar un sistema integral de gestión y monitoreo para un frigorífico inteligente, destacando la interacción entre diferentes capas tecnológicas: procesado, aplicación y base de datos. Este sistema no solo integra diversas tecnologías como MQTT, Apache Tomcat y una base de datos MariaDB, sino que también asegura la interoperabilidad entre los componentes gracias a una arquitectura modular y escalable.

En la capa de procesado, el uso de MQTT como protocolo de mensajería ha resultado esencial para la comunicación en tiempo real entre los sensores del sistema y la capa de aplicación. La incorporación de un broker MQTT ha facilitado la publicación y suscripción a eventos críticos, como alertas de anomalías, actualizaciones de inventario, y estados de sensores. La robustez de esta capa se complementa con el uso de servlets en Apache Tomcat, los cuales han proporcionado un punto de entrada seguro y eficiente para manejar las solicitudes del sistema y las operaciones de backend.

La base de datos ha sido diseñada para almacenar información clave sobre productos, usuarios, sensores, registros e inventarios. Su estructura asegura la trazabilidad de los

datos y la capacidad de realizar operaciones complejas como estadísticas de consumo o generación de alertas. Además, la integración con los componentes del sistema permite mantener la consistencia y sincronización de los datos en tiempo real.

En la capa de aplicación, se han desarrollado dos interfaces fundamentales: una aplicación móvil y una aplicación web. Estas herramientas permiten a los usuarios interactuar con el sistema de manera intuitiva, realizar un seguimiento de los productos almacenados, y recibir notificaciones sobre el estado del frigorífico y posibles anomalías. La arquitectura modular del sistema permite que estas interfaces evolucionen en el futuro, incorporando nuevas funcionalidades según las necesidades de los usuarios.

Con todo ello, se ha logrado una comunicación en tiempo real eficiente y confiable, una base de datos robusta y coherente e interfaces de usuario funcionales y fáciles de usar.

Finalmente, se concluye que el sistema desarrollado no solo es una solución funcional para la gestión de un frigorífico inteligente, sino que también sienta las bases para futuros desarrollos, como la integración con tecnologías avanzadas como machine learning para predicción de consumo o el análisis predictivo de anomalías.

5. Bibliografía

1. "Introducción a Android Studio," Android Developers, 2021. [En línea]. Disponible: <https://developer.android.com/studio/intro?hl=es-419>.
2. "Introducción a los servlets," INAP, 2020. [En línea]. Disponible: https://espaciocompartir.inap.es/v3/pluginfile.php/3699/mod_resource/content/16/Introducción.pdf.
3. "Cómo crear un servlet en Java," Rosa Marfil, 2021. [En línea]. Disponible: <https://rosamarfil.es/tutoriales/programacion/crear-un-servlet/>.
4. "Video tutorial de Servlets Java," YouTube, 2022. [En línea]. Disponible: <https://www.youtube.com/watch?v=zXAGB8m8TbA>.
5. "¿Qué es y cómo funciona Nutri-Score?," El Correo, 2021. [En línea]. Disponible: <https://www.elcorreo.com/bizkaiaadmoda/salud/nutriscore-significado-app-productos-etiqueta-alimentos-procesados-alimentacion-nutricion-comida-sana-real-nutricionista-supermercado-semaforo-nutricional-20210524124910-nt.html>.

6. "¿Qué es MQTT y cómo funciona?," Aprender Big Data, 2021. [En línea]. Disponible: <https://aprenderbigdata.com/mqtt-mosquitto/>.
7. "Creación de un servicio web RESTful," Academia Android, 2021. [En línea]. Disponible: <https://academiaandroid.com/creacion-servicio-web-restful/>.
8. "Instrucciones básicas de SQL en MariaDB," MariaDB, 2021. [En línea]. Disponible: <https://mariadb.com/kb/es/basic-sql-statements/>.
9. "Creación de una aplicación web sencilla mediante Servlets y JSP," CodeGym, 2021. [En línea]. Disponible: <https://codegym.cc/es/groups/posts/es.52.creacion-de-una-aplicacion-web-sencilla-mediante-servlets-y-jsp-parte-1->.
10. "Configuración y despliegue con Apache Tomcat," Profesores ELO, 2022. [En línea]. Disponible: http://profesores.elo.utfsm.cl/~agv/elo323/2s06/projects/CastilloGonzalez/Proyecto_RII_Tomcat.html.
11. "ServerUbicua," GitHub, 2023. [En línea]. Disponible: <https://github.com/acastillom3/ServerUbicua>.
12. "EjemplosArduino," GitHub, 2023. [En línea]. Disponible: <https://github.com/acastillom3/EjemplosArduino>.
13. "UbicuaAndroidExample," GitHub, 2023. [En línea]. Disponible: <https://github.com/acastillom3/UbicuaAndroidExample>.

Anexo I – Manual de instalación

Para instalar y configurar el proyecto correctamente, siga estos pasos detalladamente. La instalación incluye configuraciones en el servidor, despliegue de la aplicación en Tomcat y configuración inicial de la aplicación.

Paso 1: Preparación del Entorno

- Instalación de Tomcat: Asegúrese de tener Tomcat instalado en su servidor. Esto puede hacerse descargando la versión más reciente de Apache Tomcat y descomprimiéndola en su servidor.

Paso 2: Despliegue de la Aplicación

- Compilación del Proyecto: Utilice NetBeans o su IDE preferido para compilar el proyecto. Debe generar un archivo .war que es el paquete web que se desplegará en Tomcat.
- Despliegue en Tomcat: Acceda a la interfaz de gestión de Tomcat a través de `http://<IP-del-servidor>:8080/manager/html`. Desde aquí, puede subir el archivo .war usando la opción de "Desplegar" que se encuentra en la interfaz.
- Verificación del Despliegue: Una vez desplegado, Tomcat mostrará un mensaje de confirmación. Puede verificar que la aplicación está corriendo accediendo a `http://<IP-del-servidor>:8080/ServerExampleUbiComp-1.0-SNAPSHOT`, donde debería cargar la interfaz principal del proyecto.

Paso 3: Configuración del Servidor y la Base de Datos

- Conexión SSH: Conéctese al servidor donde se ha instalado Tomcat utilizando SSH con la dirección `ubicua.IP`.
- Configuración del Contexto: Acceda al archivo `context.xml` dentro de Tomcat para configurar el pool de conexiones. Ajuste los parámetros como máximo de conexiones, tiempo máximo inactivo, etc., para optimizar el rendimiento y la seguridad. Por ejemplo, puede configurar un máximo de 100 conexiones, 30 maxidle y 2000 maxthread.
- Comando de Estado: Utilice el comando `STATUS` para verificar que Tomcat está escuchando y que el servidor está operativo.

Paso 4: Funcionamiento de la Aplicación Web

- Interfaz Principal: La interfaz de usuario, definida en `index.html`, incluye botones para iniciar, parar y cambiar el modo de operación, además de un formulario para añadir productos y usuarios.
- Actualizaciones Automáticas: La página se actualiza automáticamente cada 7 segundos, actualizando también la base de datos según las operaciones realizadas.

Paso 5: Iniciación y Monitoreo

- Al iniciar la aplicación, el modo por defecto configurado por MQTT es "INSERTAR". Puede cambiarlo usando la interfaz, al igual que puede realizar las operaciones definidas de añadir productos y/o usuarios.
- Use las herramientas de monitoreo y logs del servidor para asegurarse de que todo está funcionando correctamente.

Paso 6: Ejecución en dispositivos Android:

- Para una mayor comodidad y flexibilidad, una vez que el servidor y la aplicación web están funcionando correctamente, puede optar por ejecutar la aplicación complementaria en Android Studio. Esto le permitirá interactuar con el sistema desde cualquier dispositivo Android, proporcionando acceso móvil a las funcionalidades del proyecto. Simplemente abra el proyecto en Android Studio, configure las conexiones al servidor adecuadamente y ejecute la aplicación.

Este manual asegura que el usuario pueda instalar y configurar todo el entorno necesario para el correcto funcionamiento del proyecto, considerando tanto la configuración del software como del hardware cuando sea necesario.

Anexo II – Manual de usuario de la aplicación Móvil

1. Introducción:

Este manual describe el uso de la aplicación móvil del frigorífico inteligente, una herramienta diseñada para optimizar la gestión de su frigorífico desde su teléfono. La aplicación permite realizar un seguimiento del inventario, controlar los sensores del dispositivo, recibir notificaciones de alertas y consultar estadísticas de consumo.

2. Primeros pasos:

Asegúrate de que tu dispositivo móvil y/o Android esté conectado a la red Wi-Fi para garantizar una comunicación efectiva con el frigorífico inteligente.

3. Pantalla de carga

La primera pantalla que verá al iniciar la aplicación es la de carga, donde se le da la bienvenida y se muestra un icono de carga durante aproximadamente 3 segundos, indicando que el servicio de la aplicación se está preparando para su uso.



Figura 17: Pantalla de carga

4. Pantalla principal

Tras la pantalla de carga, se le dirigirá a la pantalla de inicio, donde se muestra el logo del proyecto y dos opciones: Iniciar sesión y Registrarse. Si es un nuevo usuario,

deberá registrarse, si no, puede pasar directamente al inicio de sesión donde se le pedirán sus credenciales.

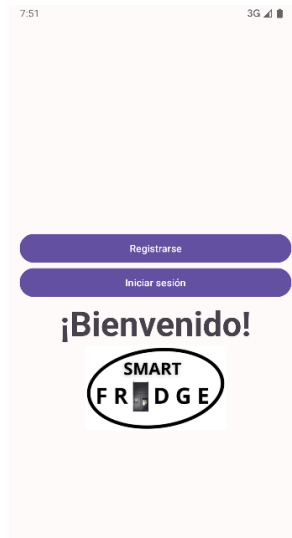


Figura 18: Pantalla principal

5. Registro de usuario:

Cuando seleccione la opción de registrarse, deberá completar los campos obligatorios que incluyen su nombre, correo electrónico y una contraseña. Es importante que la contraseña tenga un mínimo de 8 caracteres para asegurar la seguridad de su cuenta. Una vez que complete el registro de manera exitosa, recibirá una notificación confirmando que su cuenta ha sido creada correctamente. Posteriormente, podrá proceder al inicio de sesión para acceder a las funcionalidades de la aplicación.

8:36 3G

☐ Male ☐ Female

Name...

Password...

Mail...

Already registered?

Register Log in

Figura 19: Pantalla de registro

6. Inicio de sesión:

Si ya está registrado, seleccione Iniciar Sesión y proporcione su correo y contraseña. Un inicio de sesión exitoso le llevará directamente a la pantalla de visualización de inventario, de lo contrario, recibirá un mensaje de error.

SMART
FRIDGE

User: _____

Password: _____

Log in

Figura 20: Pantalla de login

7. DashboardActivity: Control y Gestión del Frigorífico

Es la interfaz principal de la aplicación una vez el usuario ha iniciado sesión. Esta pantalla ofrece una vista completa del estado actual del frigorífico, incluyendo el inventario, notificaciones de alertas y diversas funciones de control y monitoreo:

- **Inventario Actual:** Muestra una lista actualizada de todos los productos almacenados en el frigorífico, junto con información detallada como la fecha de caducidad de cada producto, facilitando así la gestión eficiente de los alimentos.
- **Campana de Notificaciones:** Ubicada en la esquina superior derecha, esta campana alerta al usuario sobre notificaciones importantes, como alertas de temperatura, humedad, o la apertura de la puerta del frigorífico. Al pulsar sobre esta campana, se muestra un mensaje tipo "toast" con los detalles de la alerta, permitiendo al usuario tomar medidas correctivas de inmediato.
- **Botones de Navegación:**
 - **Medidas Sensores:** Al seleccionar esta opción, el usuario es dirigido a otra interfaz donde puede visualizar en tiempo real los datos proporcionados por los sensores instalados en el frigorífico. Esto incluye mediciones de temperatura, humedad, detección de agua y si la puerta está abierta o cerrada, asegurando que el ambiente dentro del frigorífico es el óptimo para la conservación de alimentos.
 - **Estadísticas de Consumo:** Este botón lleva al usuario a una nueva interfaz donde puede consultar estadísticas detalladas sobre el consumo de alimentos y las alertas registradas desde una fecha específica. Esta función es esencial para usuarios que desean monitorizar y mejorar sus hábitos de consumo, ofreciendo una perspectiva clara sobre cómo interactúan con su frigorífico y los alimentos que almacenan.
 - **Crear Producto:** Facilita la entrada de nuevos productos al inventario del frigorífico. El usuario puede registrar productos manualmente, lo cual es especialmente útil para añadir alimentos caseros o aquellos que no vienen preetiquetados con RFID. Esto ayuda a mantener un control exhaustivo sobre todos los tipos de alimentos almacenados.
 - **Cerrar Sesión:** Permite al usuario cerrar su sesión y regresar a la pantalla inicial, asegurando que la información del frigorífico quede protegida y solo accesible para usuarios autorizados.

- **Modo Entrada de Alimentos:** Un interruptor en la interfaz permite al usuario activar o desactivar el modo de entrada de alimentos. Cuando está activo, cualquier producto que pase por el lector RFID se añade automáticamente al inventario. Si está desactivado, se asume que los productos son retirados del inventario. Esta funcionalidad es crucial para mantener actualizado el inventario de manera automática y eficiente.

Cada una de estas funcionalidades está diseñada para ofrecer al usuario un control total sobre su frigorífico, mejorando la gestión de alimentos y asegurando el mantenimiento de la calidad y seguridad alimentaria. La interfaz es intuitiva y fácil de usar, asegurando que incluso usuarios con poca experiencia tecnológica puedan aprovechar todas las ventajas de un frigorífico inteligente.

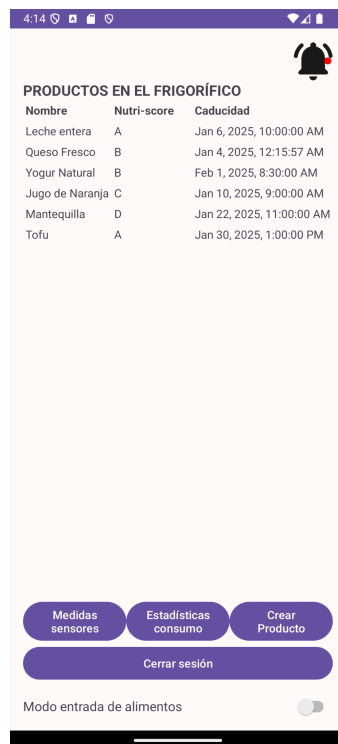


Figura 21: Pantalla de inventario

8. Medición de Sensores:

Al seleccionar “Medidas sensores”, accede a una pantalla que muestra en tiempo real los datos de los sensores del frigorífico: agua, temperatura, humedad y puerta. De esta forma podremos saber si el frigorífico está en un buen estado a través de la app.

- Temperatura: Muestra los grados a los que se encuentra el interior del frigorífico.
- Humedad: Muestra el porcentaje de humedad que se encuentra en el interior del frigorífico.
- Agua: Mostrará el valor 1 en caso de que se detecte agua en el fondo del frigorífico, lo que podría indicar que se ha descongelado algún producto o similar. Mostrará 0 en caso de no detectar agua.
- Puerta: Mostrará el valor 1 en caso de encontrarse abierta y 0 en caso contrario.

Además podremos volver a la interfaz anterior de control y gestión del frigorífico pulsando el botón “Volver”.

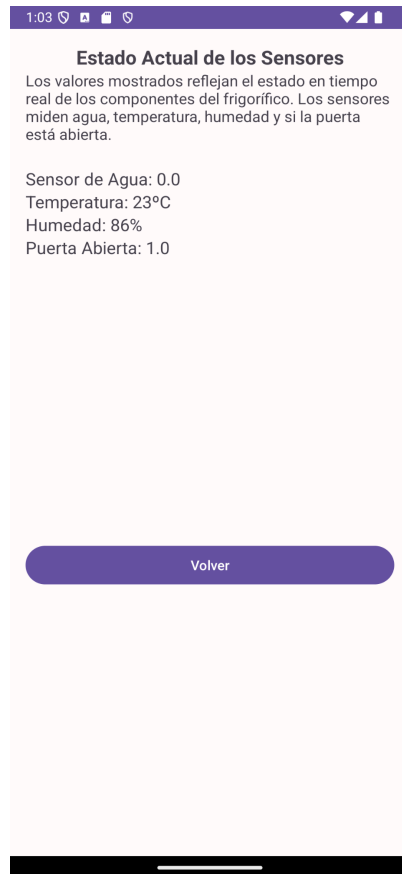


Figura 22: Pantalla de medición de sensores

9. Estadísticas de Consumo:

Esta interfaz le permite introducir una fecha específica para consultar el consumo de alimentos y las alertas registradas desde esa fecha en adelante, presentando la información en forma de estadísticas detalladas.

En detalle, la interfaz se divide en varias secciones:

- **Consumo de Alimentos:** Aquí se muestra un análisis del consumo de alimentos basado en los nutri-scores de los productos consumidos desde la fecha seleccionada. Esta sección ayuda a evaluar qué tan saludables han sido las elecciones alimentarias del usuario, presentando porcentajes de cada categoría de nutri-score.
- **Alertas de Agua y Alta Temperatura:** Se presentan estadísticas sobre las alertas detectadas relacionadas con la detección de agua y variaciones significativas de temperatura. Este seguimiento es vital para la prevención de daños o deterioro de los alimentos almacenados. Se ha decidido excluir las alertas de puerta abierta de estas estadísticas, ya que tales alertas podrían reflejar simplemente el uso normal del frigorífico por parte del usuario, sin indicar necesariamente un problema.

En la parte inferior de la interfaz, se incluye un botón "Volver". Al seleccionarlo, será redirigido a la pantalla anterior de gestión y control del frigorífico.

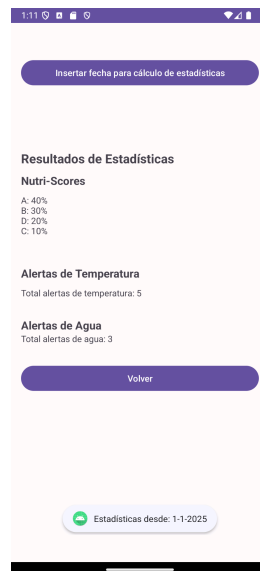


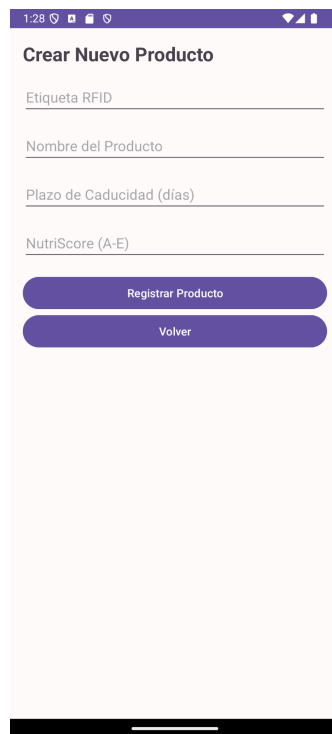
Figura 23: Pantalla de estadísticas

10. Creación de productos:

En la interfaz de "Crear Producto", usted tiene la posibilidad de registrar nuevos productos en el sistema. Esta funcionalidad es particularmente valiosa para etiquetar comidas preparadas en casa o productos que no se encuentren previamente registrados en la base de datos. Para añadir un producto, necesitará proporcionar la siguiente información:

- Código RFID: Este es el identificador único del producto.
- Nombre del Producto: El nombre que desea asignar al producto.
- Duración en Buen Estado: El tiempo que el producto puede conservarse en buen estado dentro del frigorífico.
- Nutri-Score: La calificación nutricional del producto, que ayuda a entender la calidad nutricional de manera rápida y efectiva.

Además de estos campos de entrada, la interfaz incluye un botón "Volver". Al hacer clic en este botón, usted será redirigido a la pantalla anterior, permitiéndole continuar gestionando el inventario y otras funcionalidades del sistema con facilidad.



1:28

Crear Nuevo Producto

Etiqueta RFID

Nombre del Producto

Plazo de Caducidad (días)

NutriScore (A-E)

Registrar Producto

Volver

Figura 24: Pantalla de creación de producto

11. Gestión de Alertas:

La campana de notificaciones en el Dashboard le informará sobre cualquier alerta en tiempo real, como una puerta abierta o una fuga de agua, mejorando la gestión de la seguridad y mantenimiento de su frigorífico.

12. Cierre de sesión:

Es recomendable cerrar sesión tras finalizar su interacción con la aplicación para proteger su privacidad y datos.

13. Navegación y Usabilidad:

La aplicación está diseñada para ser intuitiva y fácil de usar, con transiciones suaves entre pantallas y una interfaz clara que facilita la gestión diaria de su frigorífico.

Anexo III – Manual de Usuario de la aplicación Web

Este manual tiene como objetivo proporcionar una guía detallada sobre cómo utilizar la aplicación web asociada a nuestro sistema. A continuación, se describen los pasos y funcionalidades disponibles en la interfaz web.

Inicio de la Aplicación Web

- Acceso a la Aplicación: Abra su navegador y acceda a la dirección URL proporcionada, generalmente en el formato `http://<IP-del-servidor>:8080/ServerExampleUbiComp-1.0-SNAPSHOT`. Al ingresar, se encontrará con la página principal que incluye varios controles y una visualización de datos que inicialmente estará vacía.

Interacción con la Aplicación

- Inicio de Actualizaciones Automáticas:
 - Botón Start Interval: Para comenzar a cargar y actualizar automáticamente los datos cada 7 segundos, debe presionar el botón Start Interval. Esto activará el proceso que refresca y muestra los datos de productos, inventarios, usuarios y sensores en tiempo real.
 - Visualización de Datos: Una vez activado el intervalo, los datos empezarán a mostrarse en la parte inferior de la página. Verá tablas que listan los productos disponibles, los detalles del inventario como las fechas de caducidad de los productos, los usuarios registrados en el sistema, los valores actuales que detectan los sensores del sistema, y un registro de eventos que incluye entradas, salidas de productos y anomalías detectadas.
- Detener las Actualizaciones Automáticas:
 - Botón Stop Interval: Si desea pausar las actualizaciones automáticas, puede hacer clic en el botón Stop Interval. Esto detendrá el proceso de actualización, permitiendo que la información se mantenga estática en su última actualización.

Gestión de Productos y Usuarios

- Creación de Productos:
 - Formulario de Productos: En la parte superior de la página encontrará un formulario para añadir nuevos productos al sistema. Deberá introducir los detalles requeridos como el código RFID, el nombre del producto, la durabilidad (plazo de caducidad) y el nutri-score.
 - Botón de Envío: Una vez completado el formulario, presione el botón correspondiente para añadir el producto a la base de datos, que luego será visible en la sección de datos de productos.
- Registro de Usuarios:

- Formulario de Usuarios: Similar al de productos, existe un formulario para registrar nuevos usuarios. Complete los campos con el nombre, correo electrónico, contraseña y un identificador de inventario asociado.
- Botón de Registro: Envíe el formulario utilizando el botón proporcionado para registrar el usuario en la base de datos, y este aparecerá en la sección de usuarios registrados.

Operaciones Adicionales

- Cambio de Modo:
 - En cualquier momento, puede cambiar el modo operativo del sistema utilizando el botón Cambiar Modo, que alterna entre modos como "Insertar" y "Eliminar", adecuado para gestionar cómo se añaden o eliminan productos del inventario.

Este manual debería ayudarle a manejar eficientemente la aplicación web y aprovechar todas las funcionalidades que ofrece para una gestión óptima de los datos y operaciones del sistema.

Servidor SmartFridge

Start Interval

Stop Interval

Cambiar Modo

Modo: Esperando...

Ingrese Producto

RFID Tag

Nombre

Nutri Score

Plazo de Caducidad

Añadir Producto

Productos

RFID Tag	Nombre	Plazo de Caducidad	Nutri Score

Inventario

ID	Producto ID	Fecha de Entrada	Fecha de Caducidad

Ingrese Usuario

ID

Nombre

Correo

Password

Inventario

Ingresar Usuario

Usuarios

ID	Nombre	Correo	Password	Inventario

Sensores

ID	Tipo	Medición	Última Lectura

Registro

Figura 25: Pantalla inicial de la página web

Servidor SmartFridge

Start Interval

Stop Interval

Cambiar Modo

Modo: Esperando...

Ingreso Producto

RFID Tag

Nombre

Nutri Score

Plazo de Caducidad

Añadir Producto

Productos

RFID Tag	Nombre	Plazo de Caducidad	Nutri Score
0AJD1JCK	Producto5	26	A
0VR9FQ7P	Producto29	10	C
3DD2A8C6	Queso Fresco	5	B
4A7108AB	Yogur Natural	7	C
7LPES28Z	Producto12	16	A
A91759B3	Leche	10	B
A9ILV1PN	Producto26	9	E
AETGN87G	Producto9	9	E
ALSW1A9P	Producto14	10	C
EF2E095F	pan	2	B
EJEMPLO	EJEMPLO	11	D
EOFXQEHF	Producto21	19	E
ewfw	prueba	10	A

Figura 26: Página web tras la carga de datos

Anexo IV – Simulación de datos

Para garantizar una prueba exhaustiva y efectiva del sistema, se implementó una simulación de datos mediante un script de Python diseñado específicamente para este propósito. Este script actúa como un generador de datos que realiza inserciones automáticas y aleatorias en la base de datos, lo que permite simular un entorno de uso real de manera controlada.

El procedimiento de simulación incluye la creación de múltiples entradas en las tablas de usuarios, productos e inventarios. Para los usuarios y productos, el script genera detalles como nombres, correos electrónicos, características de productos y códigos RFID, asegurándose de que los datos sean variados y realistas. Posteriormente, a partir de los productos existentes, se simulan registros en el inventario, representando tanto la adición como la salida de productos del sistema.

Además, el script también inserta registros en la tabla de actividad del inventario, simulando eventos de entrada y salida de productos y generando alertas que podrían surgir en situaciones reales, como temperaturas inadecuadas o detección de agua. Estas inserciones permiten probar la capacidad de respuesta del sistema ante eventos simulados y ayudan a verificar la precisión y eficiencia de los mecanismos de alerta y reporte implementados en la aplicación.

Esta simulación de datos no solo es fundamental para validar la funcionalidad del sistema bajo diferentes escenarios de uso, sino que también permite identificar y corregir posibles errores y optimizar el rendimiento del sistema antes de su despliegue final.